

FILES

ADMIN STUFF

1. Check-ins due before every class
2. HW2 due on Wed, Feb 11 at 11:59pm
3. Late token balance to be shared with you as soon as I can :)
4. First exam is two weeks from today, Feb 23 in class at this time
 1. If you have accommodations, schedule these with Weingarten TODAY! I cannot do this for you or help you if you forget.
 2. The way to study is... just keep doing HW2 (and then HW3), then after that's due, take practice exams.
 3. Exam review session in class on Friday, Feb 20

FILES

FILES

On computers we have things called **files**.

Files are where we store information that the computer can still access even after the computer turns off and on again.

We have already use files before: our programs are stored in `.py` files.

For now, we can assume that the contents of files are all characters. For text files like these, we think of files as being made of a "sequence" of lines of text.

```
Is there anybody      # first line
                      # second line
out there?           # third line
```

OPEN() AND CLOSE()

To read a file, we need to create a file "object" associated with that file. We can create a variable holding a file object with the `open()` call.

```
# opens the file "filename.txt" with "r" (Reading) enabled  
example_file = open("filename.txt", "r")
```

When we are completely done with a file, we need to close it

```
example_file.close()
```

...but what do we do in between the opening and closing?

READLINE()

Once we have an open file object, we can use `readline()` to read a line from the file. TRY IT! `python first_three_lines.py hello.txt`

```
# on website as first_three_lines.py
import sys
my_file = open(sys.argv[1], "r")
for i in range(3):
    line = my_file.readline()
    print(line)
my_file.close()
```

The next time we call `readline()` we get the next line of the file. These File objects remembers our position in the file.

STRIP()

We can use the `.strip()` function on a string to remove any leading or trailing white space.

Whitespace characters are characters that just add "spacing" but don't display like typical characters: tab (`'\t'`), space (`' '`), newline (`'\n'`)

`readline()` returns a line from a file, with the newline character (`'\n'`) at the end. We can remove this newline if we call `strip()`

```
line = my_file.readline().strip()
```

SPLIT()

What if we want to get all the "words" that make-up a string?

The `split` function returns a list of strings containing all the words that have whitespace between them.

```
line = "I am 2 late"  
tokens = line.split()  
print(line) # ["I", "am", "2", "late"]
```

Note how all the elements are still strings!

PRACTICE:

Assume we have a file named `beep.boop` with the layout:

```
this file has 3 lines after this  
line 0  
line 1  
line 2
```

Write some code that can read a file like this and print out all the lines but the first.

Assume that the file can have any number instead of `3` (C12)

- You should probably use: `open(filename, "r")`, `file.readline()`, `file.close()`, `string.strip()`, `string.split()`

FUNCTIONS



DEMYSTIFYING FUNCTIONS

What's happening here?

```
import penndraw as pd
pd.rectangle(0.5, 0.5, 0.1, 0.2)
pd.run()
```

Recall:

- functions are named groups of statements
- those statements are executed when we **call** a function by name

FUNCTIONS AS NAMED GROUPS OF STATEMENTS

```
def say_hello():  
    print("Oh, hello there.")  
    print("👋")  
  
print("about to say hello.")  
say_hello()
```



```
about to say hello.  
Oh, hello there.  
👋
```

ACTIVITY: CALLING SHORT FUNCTIONS

```
def middle():  
    print(" XXXX ")  
  
def sides():  
    print("XX  XX")
```

```
middle()  
sides()  
middle()  
sides()  
middle()
```

Draw the shape that gets printed when this program is run. What is it? (S7)

```
middle()  
print(" XX")  
middle()  
sides()  
middle()
```

Draw the shape that gets printed when this program is run. What is it? (S8)

ANATOMY OF A FUNCTION

- **Function definitions** consist of the function's signature as well as a block of statements called its **body**
 - A **function signature** consists of:
 - the function's name
 - the list of parameters that it takes as input.

DISSECTING A FUNCTION

```
def multiply_two_numbers(a, b):  
    print(f"Multiplying {a} x {b}!")  
    product = a * b  
    return product
```

The **signature**:

```
def multiply_two_numbers(a, b):
```

- `def`, the function's name (`multiply_two_numbers`), a pair of parentheses, and a comma-separated list of parameters (`a` and `b`)

The **body**:

```
print(f"Multiplying {a} x {b}!")  
product = a * b  
print(product)
```

- multiple statements
- all indented one level relative to signature
- uses `a` and `b` as variables without declaring!
- can end with a `return` statement to produce a value (this example doesn't)

Choose a better name for each of the four functions below. Each function is run with a single list as its input, e.g. `M1([3, 9, 0, 14])`

```
def M1(lst):  
    smallest = lst[0]  
    for elem in lst:  
        if elem < smallest:  
            smallest = elem  
    print(smallest)
```

```
def M2(lst):  
    running_sum = 0  
    for elem in lst:  
        running_sum += 1  
    print(running_sum)
```

```
def M3(lst):  
    saved = lst[0]  
    for elem in lst:  
        if elem > saved:  
            saved = elem  
    print(saved)
```

```
def M4(lst):  
    running_sum = 0  
    for elem in lst:  
        running_sum += elem  
    print(running_sum)
```

A: `max` B: `min` C: `sum` D: `len`

RECAP: CALLING FUNCTIONS WITH INPUTS

Here is a function that takes a message and a number and prints that message that number of times.

```
def print_n_times(msg, n):  
    counter = 0  
    while counter < n:  
        print(msg)  
        counter = counter + 1
```

What happens when we call the function: `print_n_times("Hi!", 3)`?

RECAP: CALLING FUNCTIONS WITH INPUTS

- The function's **parameters** are `msg` and `n`.
 - These are names for variables that can be used in the body of the function
- The function call provides two **arguments**: `"Hi!"` and `3`.
 - These are the values that the parameter variables will take at the call start.

```
# calling print_n_times("Hi!", 3)
def print_n_times(msg, n):
    # msg = "Hi!"
    # n = 3
    counter = 0
    while counter < n: # while counter < 3:
        print(msg)    # print("Hi!")
        counter = counter + 1
```

ACTIVITY: COUNTING NUMBERS

```
def add_three_numbers(a, b, c):  
    first_two = a + b  
    last = c + first_two  
    print(last)
```

- M5: calling the function as `add_three_numbers(3, 4, 7, 9)` leads the program to immediately crash
- M6: calling the function as `add_three_numbers("three", "four", "five")` leads the program to immediately crash

A: True, B: False

ACTIVITY: WORKING TOWARDS WRITING A FUNCTION

Assuming you have a list `lst` containing a bunch of numbers, write a couple of loops that print out all of the **negative** numbers and then all of the **non-negative numbers**. (C14, but leave just a little space at the top)

e.g.

```
lst = [9, -19, 31, -13, 1, 2]  
# TODO: Your loop(s) here
```



```
-19 -13 9 31 1 2
```

You're not writing a whole function yet! Just write some lines & loops like you've been doing before.

ACTIVITY: WORKING TOWARDS WRITING A FUNCTION

Write the signature for a function that prints out all of the **negative** numbers and then all of the **non-negative numbers**. (L15)

Remember: a signature consists of a `def`, a function name, and a list of parameters the function should be called with.

ACTIVITY: WORKING TOWARDS WRITING A FUNCTION

Add a signature to the code you wrote for (C14) in order to turn it into a function that can be called.

Then, in (C16), write an example of a function call that would print out the following output:

```
-30 -14 3 19 8
```


NEW: RETURN

Function calls are themselves *expressions*, meaning that they always have a value.

- The value of a function call is determined by the value that function **returns**

`return` is keyword that serves two purposes:

- stops function execution in its tracks
- provides a value for the expression of the function call

RETURN: AN EXAMPLE

```
def multiply_two_numbers(a, b):  
    print(f"Multiplying {a} x {b}!")  
    product = a * b  
    return product
```

If we write the call `multiply_two_numbers(3, 7)` then...

```
# a = 3  
# b = 7  
print(f"Multiplying {a} x {b}!")  
product = a * b  
return product
```

*# product = 3 * 7*
return 21

...we return the value of `product`, which is `21` based on this function call. The

PRINTING VS. RETURNING

An output that's *printed* is not the same as an output that's *returned*.

- Any call to `print()` will make text appear on the screen, but it doesn't produce a value
- If a function is supposed to calculate and create some value (e.g. the product of two numbers), it must *return* that value in the function body.

FUNCTIONS THAT HAVE NO RETURN

```
def our_min(lst):  
    smallest = lst[0]  
    for elem in lst:  
        if elem < smallest:  
            smallest = elem  
    print(smallest)
```

```
def our_len(lst):  
    running_sum = 0  
    for elem in lst:  
        running_sum += 1  
    print(running_sum)
```

```
some_numbers = [1000, 3, 8]
```

```
#   3  
result = our_min(some_numbers)  
#   None  
print(result)
```

```
#   3  
result = our_len(some_numbers)  
#   None  
print(result)
```

These functions both **compute** some value and then **print** it but do not **return** it.

ADDING RETURN

```
def our_min(lst):  
    smallest = lst[0]  
    for elem in lst:  
        if elem < smallest:  
            smallest = elem  
    return smallest
```

```
def our_len(lst):  
    running_sum = 0  
    for elem in lst:  
        running_sum += 1  
    return running_sum
```

```
some_numbers = [1000, 3, 8]
```

```
#   Nothing!  
result = our_min(some_numbers)  
#   3  
print(result)
```

```
#   Nothing!  
result = our_len(some_numbers)  
#   3  
print(result)
```

These functions now **compute** some value and then **return** it but do not **print** it.

THE POINT OF NO RETURN?

`return` works as a stopping/exit point for your program. If you execute a line with `return` you will leave that function call execution.

```
def print_all_above(lst, k):  
    for elem in lst:  
        if elem > k:  
            print(elem)  
  
print_all_above([5, 10, 15], 8)
```



10 15

THE POINT OF NO RETURN?

`return` works as a stopping/exit point for your program. If you execute a line with `return` you will leave that function call execution.

```
def print_first_above(lst, k):  
    for elem in lst:  
        if elem > k:  
            print(elem)  
            return
```

```
print_first_above([5, 10, 15], 8)
```



THE POINT OF NO RETURN?

`return` works as a stopping/exit point for your program. If you execute a line with `return` you will leave that function call execution.

```
def return_first_above(lst, k):  
    for elem in lst:  
        if elem > k:  
            return elem  
  
return_all_above([5, 10, 15], 8)
```



but it does return `10`